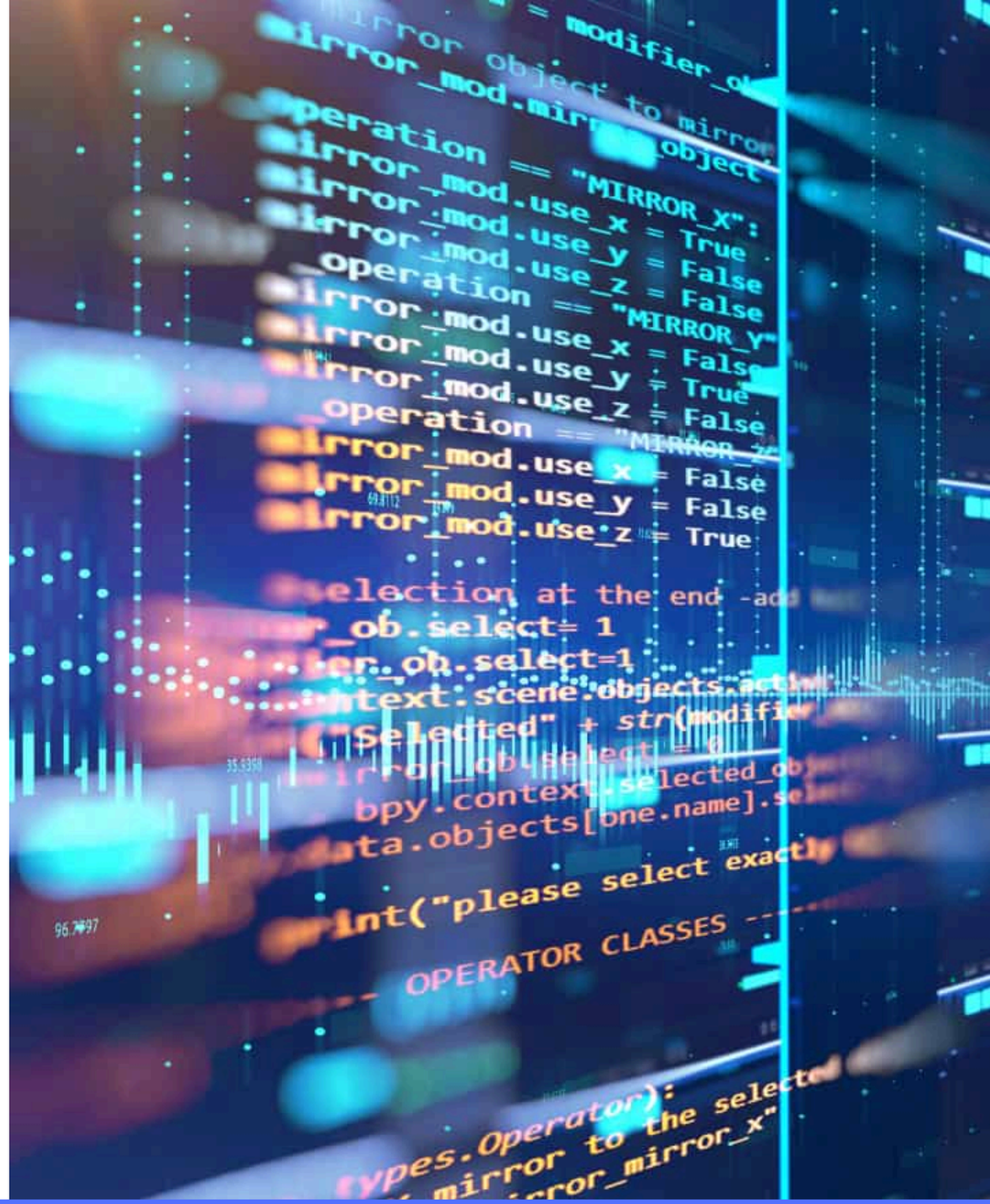
Ciência de Dados

Módulo 2: Manipulação de Dados com Pandas



INTRODUÇÃO

Por que o Pandas é tão importante?

Em Ciência de Dados, **raramente** os dados são recebidos "prontos". Em geral, as bases vêm com:

- nomes pouco intuitivos
- valores ausentes
- duplicidades
- tipos incorretos
- informações vindas de várias fontes

O **Pandas** surge como uma das principais ferramentas para **transformar dados brutos em informação útil**

Estruturas Fundamentais do Pandas

O Pandas trabalha basicamente com duas estruturas principais:

SERIES

Uma Series é uma **estrutura unidimensional**, similar a uma lista de dados organizada em uma única **coluna**

	IDHM
Fortaleza	0.754
Caucaia	0.682
Recife	0.746
Curitiba	0.778
Florianópolis	0.751

DATAFRAME

Um DataFrame é **uma estrutura bidimensional**, semelhante a uma **tabela**, com linhas e colunas

	CITY	STATE	CAPITAL	IBGE_RES_POP	AREA	GDP	IDHM
0	Fortaleza	CE	1	2452185	314.93	65151000	0.754
1	Caucaia	CE	0	325441	1228.51	9500000	0.682
2	Recife	PE	1	1537704	218.84	52017000	0.772
3	Curitiba	PR	1	1933105	434.97	98000000	NaN
4	Fortaleza	CE	1	2452185	314.93	65151000	0.754

FUNDAMENTOS

Leitura de Dados: CSV, Excel e JSON

Uma das maiores vantagens do Pandas é a facilidade para ler **diferentes formatos de arquivo**

Em projetos reais, é comum receber dados em:

- **CSV**: formato tabular simples e muito usado em análise de dados
- **Excel**: planilhas .xlsx
- **JSON**: formato muito comum em APIs e sistemas web

```python
import pandas as pd

df = pd.read_csv("brazilian_cities.csv")
df_excel = pd.read_excel("brazilian_cities.xlsx")
df_json = pd.read_json("brazilian_cities.json")
```

CITY	STATE	CAPITAL	IBGE_RES_POP	IDHM	AREA	ESTIMATED_POP	RURAL_URBAN
Abadia De Goiás	GO	0	6876	0.708	147.256	8583	Urbano
Abadia Dos Dourados	MG	0	6704	0.689	881.064	6972	Rural Adjacente
Abadiânia	GO	0	15757	0.689	1045.127	19614	Rural Adjacente

FUNDAMENTOS

Exploração dos Dados

Antes de limpar ou transformar uma base de dados, é preciso **conhecê-la**

Essa etapa ajuda a responder perguntas como:

- quantas linhas e colunas existem?
- quais são os nomes das colunas?
- quais variáveis são numéricas e quais são textuais?
- há valores ausentes?
- há sinais de problemas nos tipos de dados?

```python
# Retorna o formato do DataFrame (linhas, colunas)
df.shape

# Retorna uma lista com o nome de todas as colunas do DataFrame
df.columns.tolist()

# Exibe um resumo geral do DataFrame:
# - Tipos de dados de cada coluna
# - Quantidade de valores não nulos
df.info()

# Gera estatísticas descritivas para TODAS as colunas
# Inclui métricas como média, desvio padrão, mínimo, máximo
df.describe(include="all")
```

Identificando e Tratando Valores Ausentes

É comum encontrar valores ausentes, que podem surgir por erros de coleta, falhas de integração entre sistemas ou simplesmente ausência de informação

Nestes cenários, o primeiro passo é **identificar onde estão esses valores ausentes e entender seu impacto no dataset**. Em seguida, deve-se decidir a melhor estratégia de tratamento, que pode incluir:

- **Remover** registros incompletos
- **Substituir** valores ausentes por médias, medianas ou valores padrão
- **Preencher** com base em regras de negócio

Ignorar valores ausentes pode levar a **análises incorretas, distorções estatísticas e conclusões erradas**

```python
# Verificar valores ausentes
df.isnull()
df.isnull().sum()

# Verificar valores não nulos
df.notnull()

# Remover linhas com valores ausentes
df.dropna()

# Remover colunas com valores ausentes
df.dropna(axis=1)

# Preencher valores ausentes
df.fillna(0)
df.fillna(df.mean(numeric_only=True))

# Preencher com valor específico por coluna
df['coluna'] = df['coluna'].fillna(valor_especifico)

# Substituir valores específicos (ex: 0 como ausente)
df.replace(0, np.nan)
```

FUNDAMENTOS

Filtros e Seleção de Dados

Após a limpeza dos dados, **o próximo passo é explorar e extrair informações relevantes**. Para isso, utilizamos **filtros e seleções**, que permitem trabalhar apenas com subconjuntos específicos do dataset

Filtros são usados para **selecionar linhas com base em condições lógicas**, enquanto a seleção permite escolher colunas ou partes específicas dos dados.

Essas operações são fundamentais para responder perguntas como:

- "Quais municípios têm maior PIB?"
- "Quais registros atendem determinado critério?"
- "Quais variáveis são relevantes para análise?"

```python
# Selecionar colunas
df['coluna']
df[['coluna1', 'coluna2']]

# Selecionar linhas por índice
df.loc[0]              # por rótulo
df.iloc[0]             # por posição

# Selecionar linhas e colunas
df.loc[0:5, ['coluna1', 'coluna2']]
df.iloc[0:5, 0:2]

# Filtrar por condição
df[df['coluna'] > 10]

# Múltiplas condições
df[(df['coluna'] > 10) & (df['outra'] < 5)]
df[(df['coluna'] > 10) | (df['outra'] < 5)]

# Filtrar valores específicos
df[df['coluna'] == 'valor']
df[df['coluna'].isin(['A', 'B', 'C'])]

# Selecionar maiores/menores valores
df.nlargest(10, 'coluna')
df.nsmallest(10, 'coluna')
```

FUNDAMENTOS

Ordenação de Dados

A ordenação de dados permite **reorganizar o dataset com base em uma ou mais colunas**, facilitando a identificação de padrões, extremos e comparações.

Com a ordenação, conseguimos responder rapidamente perguntas como:

- "Quais são os maiores valores?"
- "Quais registros têm menor desempenho?"
- "Como os dados se distribuem em ordem crescente ou decrescente?

```python
# Ordenar por uma coluna (crescente - padrão)
df.sort_values('coluna')

# Ordenar por uma coluna (decrescente)
df.sort_values('coluna', ascending=False)

# Ordenar por múltiplas colunas
df.sort_values(['coluna1', 'coluna2'])

# Ordenar com critérios diferentes por coluna
df.sort_values(['coluna1', 'coluna2'], ascending=[True, False])

# Ordenar pelo índice
df.sort_index()

# Ordenar índice em ordem decrescente
df.sort_index(ascending=False)

# Selecionar maiores valores após ordenação
df.sort_values('coluna', ascending=False).head(10)
```

FUNDAMENTOS

Criação de Novas Colunas

A criação de novas colunas **permite derivar novas informações a partir dos dados existentes**

Em vez de trabalhar apenas com os dados brutos, podemos **criar métricas, indicadores e variáveis auxiliares** que facilitam a análise. Por exemplo:

- Relações entre variáveis (ex: PIB per capita)
- Indicadores (ex: densidade, proporções)
- Classificações (ex: "Alto", "Médio", "Baixo")

```python
# Criar coluna com operação simples
df['nova_coluna'] = df['coluna1'] + df['coluna2']

# Exemplo: razão entre variáveis
df['gdp_per_capita_calc'] = df['GDP'] / df['ESTIMATED_POP']

# Criar coluna com valor fixo
df['fonte'] = 'IBGE'

# Criar coluna com condição
df['categoria'] = df['GDP'].apply(lambda x: 'Alto' if x > 100000 else 'Baixo')

# Usando np.where (mais eficiente)
df['categoria'] = np.where(df['GDP'] > 100000, 'Alto', 'Baixo')

# Múltiplas condições
df['nivel'] = np.select(
    [
        df['GDP'] > 100000,
        df['GDP'] > 50000
    ],
    ['Alto', 'Médio'],
    default='Baixo'
)

# Criar coluna baseada em texto
df['estado_sigla'] = df['STATE'].str.upper()
```

FUNDAMENTOS

Agrupamentos e Agregações (GroupBy)

Os agrupamentos permitem **organizar** os dados em grupos **com base em uma ou mais variáveis**, enquanto as agregações **aplicam funções sobre esses grupos para gerar resumos**

O processo segue a lógica:
dividir → aplicar → combinar

```python
# Agrupar por uma coluna
df.groupby('coluna')

# Média por grupo
df.groupby('coluna')['valor'].mean()

# Soma por grupo
df.groupby('coluna')['valor'].sum()

# Múltiplas agregações
df.groupby('coluna')['valor'].agg(['mean', 'sum', 'min', 'max'])

# Agrupar por múltiplas colunas
df.groupby(['coluna1', 'coluna2'])['valor'].mean()

# Resetar índice após groupby
df.groupby('coluna')['valor'].mean().reset_index()

# Renomear colunas após agregação
df.groupby('coluna')['valor'].mean().reset_index(name='media_valor')

# Aplicar múltiplas funções em várias colunas
df.groupby('coluna').agg({
    'valor1': 'mean',
    'valor2': 'sum'
})
```

FUNDAMENTOS

Estatística Descritiva Básica

A estatística descritiva tem como objetivo r**esumir e descrever os dados por meio de métricas numéricas**, permitindo uma compreensão rápida do seu comportamento

Com essas medidas, conseguimos identificar:

- Tendência central (onde os dados se concentram)
- Dispersão (o quanto os dados variam)
- Valores extremos (outliers)

No Pandas, diversas métricas estatísticas podem ser calculadas de forma simples e direta, tornando essa etapa rápida e eficiente

```python
# Resumo geral do dataset
df.describe()

# Incluir todas as colunas (inclusive categóricas)
df.describe(include='all')

# Média
df['coluna'].mean()

# Mediana
df['coluna'].median()

# Moda
df['coluna'].mode()

# Variância
df['coluna'].var()

# Desvio padrão
df['coluna'].std()

# Valores mínimos e máximos
df['coluna'].min()
df['coluna'].max()

# Quantis
df['coluna'].quantile(0.25)
df['coluna'].quantile(0.5)
df['coluna'].quantile(0.75)

# Contagem de valores únicos
df['coluna'].nunique()

# Frequência de valores
df['coluna'].value_counts()
```

MÓDULO 2

Exercícios

- Liste os 10 municípios com maior GDP_CAPITA.
- Filtre apenas as capitais e compare IDHM e GDP.
- Crie uma coluna de motos por habitante.
- Agrupe por STATE e descubra quantos municípios existem em cada estado no recorte analisado.
- Agrupe os dados por estado STATE e calcule o PIB total (GDP) por estado.
- Verifique quais municípios têm HOTELS maior que zero e UBER igual a zero.
- Ordene os municípios por ESTIMATED_POP e observe os maiores valores.
- Compare a média de IDHM por estado.
- Calcule o PIB médio GDP por estado e ordene do maior para o menor.
- Exiba qual estado possui a maior média de IDHM.

Conclusão e Próximos Passos

O próximo passo é avançar para técnicas de visualização e análise mais sofisticadas

Até o módulo 3!